ARQUITECTURAS PARALELAS Y COMPILADORES II CICLO 2026

Proyecto I: convolución con vectorización e hilos

Profesor:	Ignacio Gómez
Modalidad:	individual
Valor:	35 % de la nota final
Lenguaje:	C (estándar C11) con pthreads, en la computadora de cada estudiante
Evaluación:	preguntas en una cita individual (100 % de la nota del proyecto)
Código final:	viernes 6 de noviembre de 2026, 23:59

1. De qué se trata

Usted va a escribir en C un programa que aplica tres filtros a imágenes en escala de grises. Primero lo hace de la forma más simple, y después lo acelera de dos maneras vistas en el curso: logrando que el compilador vectorice el bucle interno y repartiendo el trabajo entre varios hilos con pthreads. Cada versión se mide y se compara con la anterior.

Todo el código es suyo y todo es en C. Se le entregan las imágenes de prueba y un archivo `pgm.h` para leerlas y escribirlas. La nota sale de las preguntas que le hace el profesor sobre su código en una cita individual.

2. Cómo se aplica un filtro

Una imagen en escala de grises es una matriz de números entre 0 (negro) y 255 (blanco). Un filtro es una matriz pequeña de coeficientes, de 3×3 o de 5×5. Para calcular cada píxel de la imagen de salida se hace lo siguiente:

1. Se coloca el filtro centrado sobre el píxel, en la imagen de entrada. Eso define una *ventana* del mismo tamaño que el filtro.
2. Cada valor de la ventana se multiplica por el coeficiente del filtro que le queda encima, y se suman todos esos productos.
3. Si la suma es negativa, se le quita el signo. Si es mayor que 255, se deja en 255.
4. Se redondea al entero más cercano y ese es el valor del píxel de salida.

Los bordes. Cuando el píxel está cerca de la orilla, parte de la ventana cae fuera de la imagen. Qué valor se usa en esos casos se explica más abajo, en «Las orillas».

Escrito como fórmula, con r igual a la mitad del tamaño del filtro ($r = 1$ para 3×3, $r = 2$ para 5×5):

$$O[y][x] \;=\; \text{mín}\left(255,\; \left|\sum_{i=-r}^{r}\sum_{j=-r}^{r} K[i+r][j+r] \cdot I[y+i][x+j]\right|\right)$$

donde y es la fila y x la columna, ambas contando desde 0.

> **Para que su resultado sea idéntico al de referencia:** use `float` para los coeficientes y para la suma; sume recorriendo la ventana por filas (de arriba hacia abajo y, dentro de cada fila, de izquierda a derecha); y redondee con `(unsigned char)(suma + 0.5f)`. Compile siempre con `-ffp-contract=off`.

La imagen de ejemplo

Los ejemplos de las secciones siguientes usan esta imagen de 5×5, que viene en el material como `prueba.pgm`. Tiene un borde vertical: a la izquierda es oscura (10) y en la columna 3 aparece un valor claro (50).

$$I = \begin{bmatrix} 10 & 10 & 10 & 50 & 20 \\ 10 & 10 & 10 & 50 & 20 \\ 10 & 10 & 20 & 50 & 20 \\ 10 & 10 & 10 & 50 & 20 \\ 10 & 10 & 10 & 50 & 20 \end{bmatrix}$$

Las orillas

Cuando la ventana necesita un píxel que está fuera de la imagen, se usa el píxel de la orilla más cercana. La regla es la misma para filas y para columnas, y se aplica a cada índice por separado:

- Si el índice es menor que 0, se usa 0 (la primera fila o columna).
- Si el índice es mayor que el último, se usa el último ($H - 1$ para filas, $W - 1$ para columnas).
- Si está dentro de la imagen, se usa tal cual.

Por ejemplo, en una imagen de 5 filas, la fila -1 y la fila -2 se cambian por la fila 0, y la fila 5 y la fila 6 se cambian por la fila 4. En una esquina se corrigen las dos cosas: el píxel $(-1, -1)$ se cambia por el $(0, 0)$. En C, para cada índice:

```
int f = y + i;   if (f < 0) f = 0;   if (f > H - 1) f = H - 1;
int c = x + j;   if (c < 0) c = 0;   if (c > W - 1) c = W - 1;
```

Una forma de verlo es que la imagen se extiende hacia afuera repitiendo su orilla. Así queda la imagen de ejemplo extendida dos píxeles por lado, que es lo que necesita el filtro de 5×5 (en gris lo que se agrega; en negro la imagen original):

$$\begin{bmatrix} 10 & 10 & 10 & 10 & 10 & 50 & 20 & 20 & 20 \\ 10 & 10 & 10 & 10 & 10 & 50 & 20 & 20 & 20 \\ 10 & 10 & 10 & 10 & 10 & 50 & 20 & 20 & 20 \\ 10 & 10 & 10 & 10 & 10 & 50 & 20 & 20 & 20 \\ 10 & 10 & 10 & 10 & 20 & 50 & 20 & 20 & 20 \\ 10 & 10 & 10 & 10 & 10 & 50 & 20 & 20 & 20 \\ 10 & 10 & 10 & 10 & 10 & 50 & 20 & 20 & 20 \\ 10 & 10 & 10 & 10 & 10 & 50 & 20 & 20 & 20 \\ 10 & 10 & 10 & 10 & 10 & 50 & 20 & 20 & 20 \end{bmatrix}$$

El filtro de 3×3 solo usa un píxel extra por lado; el de 5×5 usa dos.

3. Filtro 1: blur (desenfoque)

$$K_{\text{blur}} = \frac{1}{9}\begin{bmatrix}1 & 1 & 1\\ 1 & 1 & 1\\ 1 & 1 & 1\end{bmatrix} \qquad \text{en C, cada coeficiente es } \texttt{1/9.f}$$

Todos los coeficientes son iguales, así que el resultado es el promedio de los 9 píxeles de la ventana. La imagen se suaviza.

Ejemplo, píxel del centro $O[2][2]$. La ventana son las filas 1 a 3 y las columnas 1 a 3:

$$\begin{bmatrix}10 & 10 & 50\\ 10 & 20 & 50\\ 10 & 10 & 50\end{bmatrix} \quad\Rightarrow\quad \frac{10+10+50+10+20+50+10+10+50}{9} = \frac{220}{9} = 24.44 \quad\Rightarrow\quad O[2][2] = 24$$

Ejemplo en el borde, $O[0][3]$. La ventana necesita la fila -1, que no existe, así que se usa la fila 0 (ver «Las orillas»):

$$\begin{bmatrix}10 & 50 & 20\\ 10 & 50 & 20\\ 10 & 50 & 20\end{bmatrix} \quad\Rightarrow\quad \frac{240}{9} = 26.67 \quad\Rightarrow\quad O[0][3] = 27$$

Resultado completo:

$$O_{\text{blur}} = \begin{bmatrix}10 & 10 & 23 & 27 & 30\\ 10 & 11 & 24 & 28 & 30\\ 10 & 11 & 24 & 28 & 30\\ 10 & 11 & 24 & 28 & 30\\ 10 & 10 & 23 & 27 & 30\end{bmatrix}$$

4. Filtro 2: gauss (desenfoque gaussiano)

$$K_{\text{gauss}} = \frac{1}{273}\begin{bmatrix}1 & 4 & 7 & 4 & 1\\ 4 & 16 & 26 & 16 & 4\\ 7 & 26 & 41 & 26 & 7\\ 4 & 16 & 26 & 16 & 4\\ 1 & 4 & 7 & 4 & 1\end{bmatrix} \qquad \text{en C: } \texttt{1/273.f},\ \texttt{4/273.f},\ \texttt{7/273.f},\ \ldots$$

También es un promedio, pero ponderado: el centro pesa más (41) y las esquinas casi nada (1). Suaviza la imagen de forma más natural que el blur. La ventana es de 5×5, así que $r = 2$.

Ejemplo, píxel del centro $O[2][2]$. La ventana es la imagen completa. Se multiplica cada valor por su

peso, fila por fila:

$$\begin{aligned}
&\text{fila 0:} && 1{\cdot}10 + 4{\cdot}10 + 7{\cdot}10 + 4{\cdot}50 + 1{\cdot}20 = 340\\
&\text{fila 1:} && 4{\cdot}10 + 16{\cdot}10 + 26{\cdot}10 + 16{\cdot}50 + 4{\cdot}20 = 1340\\
&\text{fila 2:} && 7{\cdot}10 + 26{\cdot}10 + 41{\cdot}20 + 26{\cdot}50 + 7{\cdot}20 = 2590\\
&\text{fila 3:} && \text{igual que la fila } 1 = 1340\\
&\text{fila 4:} && \text{igual que la fila } 0 = 340
\end{aligned}$$

$$\frac{340 + 1340 + 2590 + 1340 + 340}{273} = \frac{5950}{273} = 21.79 \quad \Rightarrow \quad O[2][2] = 22$$

En el código cada coeficiente ya viene dividido entre 273, así que no se divide al final: se suman directamente los productos.

Ejemplo en la esquina, $O[0][4]$. Aquí faltan dos filas arriba y dos columnas a la derecha. Las filas que se necesitan son $-2, -1, 0, 1, 2$, que se convierten en $0, 0, 0, 1, 2$; las columnas son $2, 3, 4, 5, 6$, que se convierten en $2, 3, 4, 4, 4$. La ventana queda así, y se multiplica por los pesos igual que antes:

$$\begin{bmatrix} 10 & 50 & 20 & 20 & 20 \\ 10 & 50 & 20 & 20 & 20 \\ 10 & 50 & 20 & 20 & 20 \\ 10 & 50 & 20 & 20 & 20 \\ 20 & 50 & 20 & 20 & 20 \end{bmatrix} \quad \Rightarrow \quad \frac{450 + 1760 + 2850 + 1760 + 460}{273} = \frac{7280}{273} = 26.67 \quad \Rightarrow \quad O[0][4] = 27$$

Resultado completo:

$$O_{\text{gauss}} = \begin{bmatrix} 10 & 13 & 21 & 29 & 27 \\ 10 & 13 & 21 & 29 & 27 \\ 10 & 13 & 22 & 30 & 27 \\ 10 & 13 & 21 & 29 & 27 \\ 10 & 13 & 21 & 29 & 27 \end{bmatrix}$$

5. Filtro 3: sobel (detección de bordes)

$$K_{\text{sobel}} = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

Este filtro resta la columna de la izquierda a la de la derecha. Si la imagen no cambia de izquierda a derecha, da 0 (negro). Donde hay un cambio fuerte, da un valor alto (blanco). Sirve para encontrar bordes verticales. Es el único de los tres que puede dar negativo; por eso se le quita el signo.

Ejemplo, píxel del centro $O[2][2]$. Misma ventana que en el blur:

$$\begin{aligned}
&\text{fila 1:} && -1{\cdot}10 + 0{\cdot}10 + 1{\cdot}50 = 40\\
&\text{fila 2:} && -2{\cdot}10 + 0{\cdot}20 + 2{\cdot}50 = 80\\
&\text{fila 3:} && -1{\cdot}10 + 0{\cdot}10 + 1{\cdot}50 = 40
\end{aligned}$$

$$40 + 80 + 40 = 160 \quad \Rightarrow \quad O[2][2] = 160$$

Ejemplo con resultado negativo, $O[2][4]$. La columna 5 no existe, así que se usa la columna 4 (ver «Las orillas»). La imagen baja de 50 a 20 de izquierda a derecha:

$$\begin{bmatrix} 50 & 20 & 20 \\ 50 & 20 & 20 \\ 50 & 20 & 20 \end{bmatrix} \quad \Rightarrow \quad (-50+20)+2(-50+20)+(-50+20) = -120 \quad \Rightarrow \quad O[2][4] = 120$$

Ejemplo cuando pasa de 255. En un borde de negro a blanco la suma se sale del rango:

$$\begin{bmatrix} 0 & 0 & 255 \\ 0 & 0 & 255 \\ 0 & 0 & 255 \end{bmatrix} \quad \Rightarrow \quad 255 + 2 \cdot 255 + 255 = 1020 \quad \Rightarrow \quad O = 255$$

Con sobel la suma puede llegar hasta 1020 o bajar hasta -1020. Primero se quita el signo y después se recorta a 255, así que todo borde fuerte sale blanco, suba o baje. Este paso no se puede omitir: convertir a `unsigned char` un valor mayor que 255 da un resultado sin sentido.

Resultado completo:

$$O_{\text{sobel}} = \begin{bmatrix} 0 & 0 & 160 & 40 & 120 \\ 0 & 10 & 160 & 30 & 120 \\ 0 & 20 & 160 & 20 & 120 \\ 0 & 10 & 160 & 30 & 120 \\ 0 & 0 & 160 & 40 & 120 \end{bmatrix}$$

6. Material que se entrega

- `pgm.h`: funciones para leer y escribir imágenes (`leer_pgm`, `escribir_pgm`), medir tiempo (`ahora`), sacar la mediana de varios tiempos (`mediana`) y calcular el checksum de una imagen (`checksum`).
- `prueba.pgm`: la imagen de 5×5 de los ejemplos. Su programa debe dar exactamente las tres matrices de resultado de arriba.
- `img2048.pgm` e `img4096.pgm`: las imágenes grandes con las que se mide el tiempo.

El checksum es un número que resume toda la imagen de salida. Si su programa da el mismo checksum que la tabla, su imagen es idéntica a la de referencia:

	img2048	**img4096**
`blur`	15228573027849069044	14133797607195559027
`gauss`	6704049370287127316	11810212636788029950
`sobel`	13749065433343460504	15895088315003003173

7. Las versiones

V0. Versión simple

Los tres filtros tal como se explicaron arriba, sin ninguna optimización. Compile sin vectorizar: `-O2 -fno-tree-vectorize` (gcc) o `-O2 -fno-vectorize -fno-slp-vectorize` (clang). Todas las demás versiones se comparan contra esta.

V1. Vectorizada por el compilador
Reorganice el código para que el compilador vectorice el bucle que recorre las columnas. Puede pedirle al compilador que le diga si lo logró: `-fopt-info-vec` en gcc, `-Rpass=loop-vectorize` en clang.

V2. Con hilos
Parta de V0 y reparta las filas de la imagen entre varios hilos con `pthread_create` y `pthread_join`. Pruebe con 1, 2, 4 hilos y con el número de núcleos de su computadora.

V3. Hilos y vectorización
Combine V1 y V2.

8. Compilar y medir

Cada estudiante trabaja en su propia computadora:

Equipo	Compilar con
Linux o WSL	`gcc -std=c11 -O3 -march=native -ffp-contract=off -pthread`
macOS	`clang -std=c11 -O3 -mcpu=native -ffp-contract=off -pthread`
Windows	instale WSL y use la primera fila

Para cada versión, filtro e imagen grande, corra el filtro 5 veces y anote la mediana del tiempo. Con eso arme una tabla y calcule el speedup de cada versión respecto a V0 (tiempo de V0 dividido entre el tiempo de la versión). Anote también el modelo de su procesador y cuántos núcleos tiene.

9. Evaluación

La nota del proyecto (100 %) sale de las preguntas que le hace el profesor en una cita individual. Las citas se programan después de la entrega del código; las fechas y los horarios se anuncian más adelante.

En la cita usted trae su computadora con el código tal como quedó en la entrega final, lo compila y lo corre, y responde preguntas sobre cómo lo hizo y sobre los tiempos que obtuvo. Durante la cita no se pueden usar herramientas de IA. El código no se califica por separado: lo que se evalúa es que usted entienda lo que escribió y pueda explicarlo.

10. Fechas y reglas

Qué	Fecha	Detalle
Repositorio	viernes 16 de octubre	Enviar el enlace en el formulario del curso.
Código final	viernes 6 de noviembre, 23:59	Las cuatro versiones y la tabla de tiempos en el repositorio.
Citas	por anunciar	Individuales.

- El proyecto es individual.
- Todo se programa en C (C11) con pthreads. No se permiten OpenMP, intrínsecos, ensamblador ni bibliotecas que apliquen los filtros.
- Haga commits a medida que avanza. Las consultas se atienden en clase o por el canal del curso.